

Fonctionnement des Ordinateurs

TP2 - Programmation en langage d'assemblage MIPS

B. QUOTIN
Faculté des Sciences
Université de Mons

Résumé

L'objectif de ce TP est de renforcer votre compréhension du jeu d'instructions MIPS. Il s'agit de bien comprendre le rôle des différentes instructions : arithmétiques/logiques, load/store, sauts, branchements conditionnels et appels de procédures.

Il vous est proposé à cet effet d'écrire un certain nombre de courts programmes en langage d'assemblage MIPS et de les exécuter dans un simulateur appelé SPIM ou QTSPIM développé par James Larus de Microsoft Research.

Table des matières

1	Démarrer avec SPIM	1
1.1	Prise en main de QTSPIM	1
1.2	Votre premier programme	2
1.3	Points d'arrêt (<i>breakpoints</i>)	3
1.4	Langage machine et codes d'instructions	3
2	Exercices introductifs	4
2.1	Boucle à 5 itérations	4
2.2	Affichage d'un entier	5
2.3	Affichage d'une chaîne de caractères	5
2.4	Somme de deux entiers	7
2.5	Lecture d'entiers et tests de conditions	7
2.6	Conversion en binaire	7
3	Exercices sur les appels de fonctions	8
3.1	Mécanisme d'appel de fonction	8
3.2	Sauvegarde de l'adresse de retour sur la pile	8
3.3	Passage d'arguments et résultat	8
3.4	Manipulation de chaînes de caractères	8
3.5	Multiplication... récursive	9

4	Exercices sur les tableaux	10
4.1	Recherche dans un tableau	10
4.2	Structure de contrôle <code>switch</code>	10
4.3	Conversion en hexadécimal	11
5	Idées d'exercices supplémentaires	13
5.1	Nombre variable d'arguments	13
5.2	Tableaux d'entiers	13
5.3	Afficher une chaîne de caractères	13
5.4	Factorielle	13
5.5	Attaque par buffer overflow	13
5.6	Combinaisons	14
6	Exercices sur les entrées/sorties	15
6.1	Entrées-sorties mappées en mémoire	15
6.2	Utilisation d'un timer	15
A	Annexes	17
A.1	Langage d'assemblage	17
A.2	Directives de l'assembleur	17
A.3	Architecture MIPS	18
A.4	Encodage des instructions MIPS	19
A.5	Appels système	20

1 Démarrer avec SPIM

Le simulateur que nous allons utiliser lors de ces travaux pratiques est QTSPIM, une version graphique du simulateur SPIM développé par James Larus de Microsoft Research. Ce simulateur est déjà installé dans les salles informatiques, mais vous pouvez l'installer sur votre propre ordinateur. Il est disponible gratuitement à l'adresse suivante : <https://sourceforge.net/projects/spimsimulator/files/>.

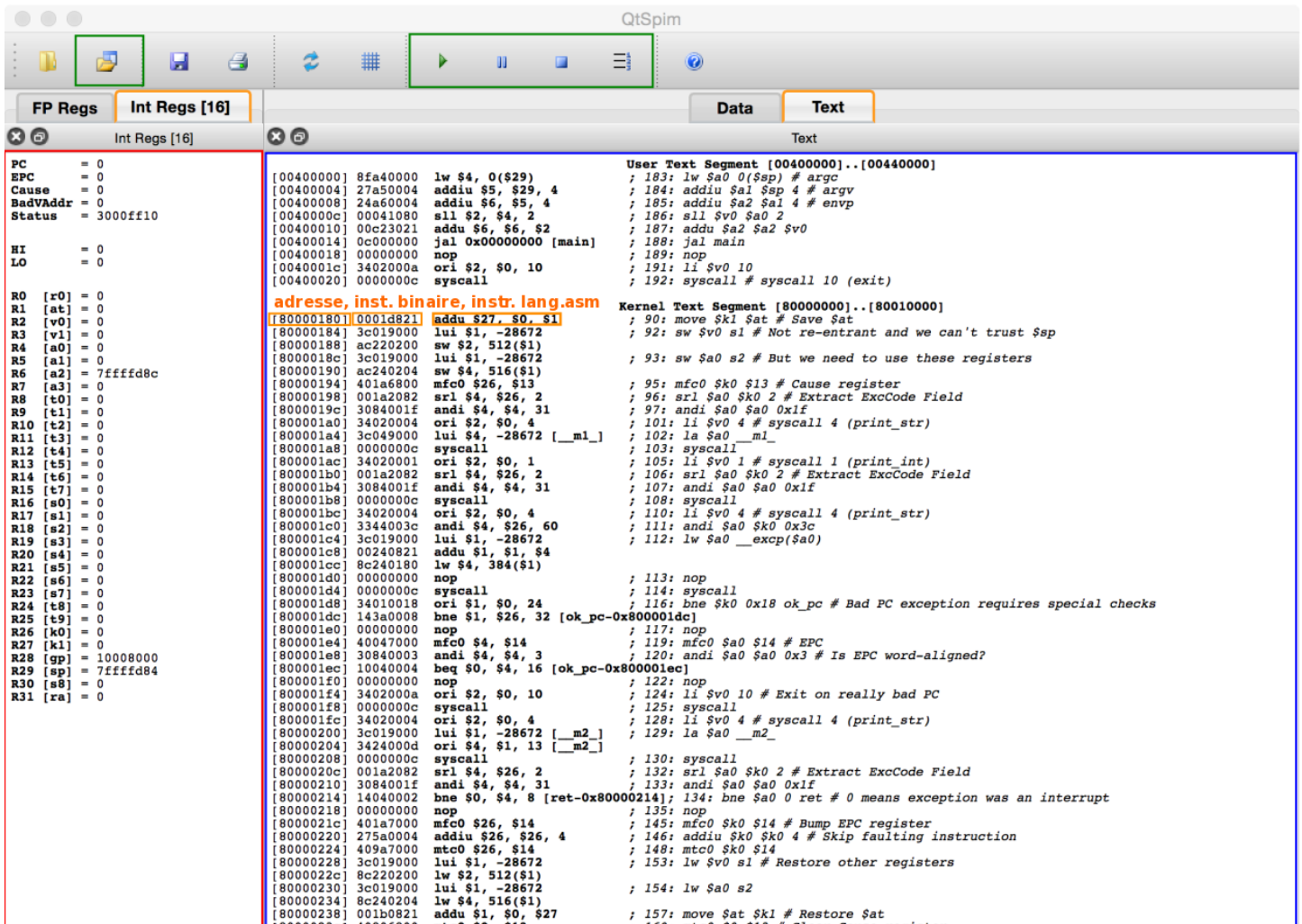
1.1 Prise en main de QTSPIM

Une capture de l'interface graphique de QTSPIM est présentée à la Figure 1. A gauche, dans le **cadre rouge**, le contenu des registres est mis à jour en temps-réel, au fur et à mesure de l'exécution des instructions. Il est possible d'y consulter la valeur de tous les registres discutés lors de ce cours : *program counter* (PC), banque de registres GPR (R_i), registres LO et HI.

Au centre, dans le **cadre bleu**, le contenu de la mémoire est accessible. Les onglets "Data" et "Text" permettent de consulter respectivement la mémoire de données ou d'instructions. Dans la mémoire d'instructions, visible sur la figure, chaque ligne représente une instruction et contient son emplacement en mémoire (adresse), son code binaire sur 32 bits, et son expression en langage d'assemblage. La partie mémoire d'instructions est découpée en deux zones, celle destinée aux programmes utilisateurs que nous allons écrire (*User Text Segment*) se situe entre les adresses 0x00400000 et 0x00440000, tandis que la mémoire réservée au système (*Kernel Text Segment*) se situe entre les adresses 0x80000000 et 0x80010000. Le simulateur démarre l'exécution des programmes à l'adresse 0x00400000.

Dans la partie haute de l'écran, plusieurs boutons importants sont entourés de **cadres verts** : ré-initialiser et charger un programme

FIGURE 1 – Fenêtre principale de QTSPIM.



Memory and registers cleared
 Loaded: /var/folders/dd/446xwd3x2j1y2nr9x9yyy_w0000gn/T/QtSpim.BMJ926
 SPIM Version 9.1.15 of April 26, 2015

Bouton	Description	Bouton	Description
	Charger un fichier.		Exécuter le programme jusqu'à sa fin.
	Charger un fichier et ré-initialiser le simulateur.		Mettre le programme en pause.
	Remettre à zéro les registres.		Arrêter l'exécution du programme.
	Ré-initialiser le simulateur.		Avancer d'une instruction.

TABLE 1 – Boutons de QTSPIM.

() , lancer l'exécution () , mettre l'exécution en pause () , arrêter l'exécution () et avancer d'une seule instruction () . La Table 1 fournit une brève description du rôle de chaque bouton.

1.2 Votre premier programme

Pour découvrir le fonctionnement de SPIM, votre première mission consiste à créer un programme MIPS minimal, à le faire exécuter *pas-à-pas* par le simulateur et à observer les informations que le simulateur peut vous fournir durant l'exécution. SPIM supporte directement le langage d'assemblage MIPS. Il se charge de convertir un programme en langage d'assemblage vers le langage machine et de charger les données et instructions définies dans le programme vers les zones mémoire correspondantes. Les particularités du langage d'assemblage supporté par le simulateur sont résumées à l'annexe A.1.

Le premier programme à réaliser exécute une boucle sans fin. Il est illustré ci-dessous. Le point d'entrée du programme est défini par le label `main`. **⚠ Tous vos programmes doivent commencer au label `main`.** Si ce label n'est pas défini, le simulateur générera une erreur. Le programme est composé de deux instructions exécutées séquentiellement. La première instruction, `nop` (*no-operation*) ne fait rien. La seconde instruction, `j main` (*jump*), effectue un branchement inconditionnel vers le label `main`.

```
1 | main:
2 |     nop
3 |     j      main
```

Pour pouvoir exécuter le programme, il faut d'abord le copier ou le re-transcrire dans un fichier. Vous pouvez utiliser à cet effet votre éditeur de texte préféré (p.ex. `gedit`, `atom` ou `vim`). Nommez votre fichier `spim-loop.s`.

Utilisez ensuite le bouton  pour ré-initialiser le simulateur et charger le programme `spim-loop.s`. SPIM s'occupe de lire le code en langage d'assemblage (texte) et de le convertir en langage machine (mots binaires). Vous pouvez observer que votre programme est chargé dans la mémoire d'instructions à l'adresse `0x00400024`. Vous devriez retrouver le code de votre programme dans la mémoire d'instructions du simulateur. Plusieurs informations sont fournies pour chaque instruction : l'adresse à laquelle l'instruction se situe, le code binaire de l'instruction (langage machine), son code en langage d'assemblage, et le code initial dans le programme que vous avez écrit. Cela permet notamment de voir comme vos instructions ou pseudo-instructions sont traduites par le simulateur.

Adresse	Langage Machine	Langage Assemblage	Programme d'origine
0x00400024	0x00000000	<code>nop</code>	; 2: <code>nop</code>
0x00400028	0x08100009	<code>j 0x00400024 [main]</code>	; 3: <code>j main</code>

Lancez l'exécution de ce programme avec le bouton  . Le simulateur est maintenant en train d'exécuter votre programme, mais celui-ci ne se terminera jamais car il effectue une boucle infinie. Pour interrompre l'exécution, utilisez le bouton  . Vous devriez constater que le programme est arrêté sur l'une des deux instructions de votre programme (vérifiez la valeur du registre PC). Vous pouvez continuer à exécuter le programme pas à pas en utilisant le bouton  . Vous devriez observer une exécution "en boucle" : la valeur du registre PC (*program counter*) doit alterner entre `0x00400024` et `0x00400028`.

L'étudiant attentif aura remarqué que lorsque SPIM est (ré-)initialisé, la valeur du registre PC vaut `0x00400000` et non l'adresse `0x00400024` qui correspond au label `main`. Qu'est-ce qui explique cette différence ? **Le point d'entrée de votre programme (`main`) est considéré comme une fonction.** Le simulateur ajoute des instructions supplémentaires à votre programme qui sont notamment chargées d'appeler cette fonction à l'aide d'une instruction `jal main`. Par ailleurs, si votre fonction `main` se termine et retourne à l'appelant (avec `jr $ra`), le code ajouté par SPIM met fin au programme en effectuant un appel système 10 (voir `syscall` en Section A.5).

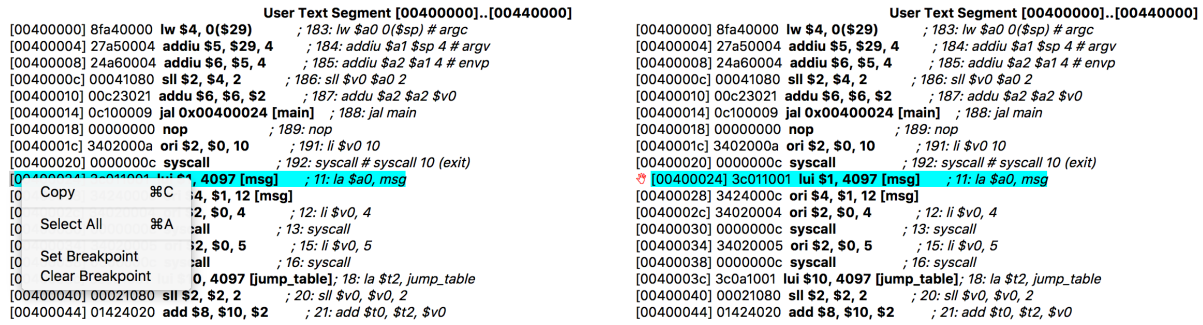


FIGURE 2 – Définir un « point d'arrêt » avec SPIM.

1.3 Points d'arrêt (breakpoints)

Une fonctionnalité importante de SPIM est celle des « points d'arrêt » (*breakpoints*). Il s'agit d'un moyen d'arrêter le simulateur lorsqu'il s'apprête à exécuter une instruction située à une adresse particulière.

La Figure 2 illustre comment définir un breakpoint à l'adresse de `main`, c'est-à-dire `0x00400024`. Il suffit d'effectuer un « clic droit » sur la ligne de l'instruction en question. Un menu contextuel apparaît dans lequel il faut sélectionner *Set Breakpoint*. Une petite **main rouge** apparaît à la gauche de l'instruction sur laquelle un point d'arrêt a été défini. A chaque fois que le simulateur arrive à l'adresse d'un breakpoint, il arrête l'exécution. Il est alors possible d'observer l'état des registres et de continuer l'exécution du programme, par exemple en avançant pas-à-pas avec .

Une première utilisation des breakpoints est le passage des instructions ajoutées par SPIM à votre programme. Avec un breakpoint défini sur `main`, il suffit de lancer l'exécution avec . Dès que le simulateur arrive à l'adresse du label `main`, il s'arrête et il est possible de poursuivre l'exécution pas-à-pas...

1.4 Langage machine et codes d'instructions

Le processeur n'exécute pas directement votre code en langage d'assemblage. Celui-ci a été traduit en langage machine, sous forme d'une suite de mots binaires encodant les instructions à exécuter. Vous pouvez observer les codes binaires des instructions dans la partie centrale de la fenêtre du simulateur, dans l'onglet *Text*. Pour rappel, toutes les instructions MIPS sont encodées sur 32-bits, mais il existe plusieurs formats d'instructions : **R**, **I** et **J**. Ces formats sont rappelés en Section A.4. Il est possible de décoder manuellement les instructions, comme le processeur le ferait. A cette fin, il suffit d'identifier d'abord l'*opcode* et sur base de celui-ci, déterminer le format (R, I ou J) de l'instruction. Il est alors possible d'extraire les opérandes de l'instruction.

Questions. Notez ci-dessous les codes des instructions `nop` et `j main` du programme `spim-loop.s`. Pour chacune des instructions, commencez par identifier l'*opcode*; déterminez ensuite le format de l'instruction et récupérez les opérandes. Pour l'instruction `j main`, calculez l'adresse de branchement et vérifiez qu'elle correspond à l'adresse du label `main`.

Q1) Décodage de l'instruction `nop`.

.....

.....

.....

Q2) Décodage de l'instruction `j main`.

.....

.....

.....

2 Exercices introductifs

Cette section propose une série d'exercices de programmation en langage d'assemblage, de difficulté croissante. Il est recommandé de les résoudre dans l'ordre dans lequel ils sont proposés. Il est également recommandé d'adopter dès le départ de bonnes pratiques en matière de formatage du code. Indenter les instructions par rapport aux labels rendra vos programmes plus faciles à lire. Des lignes vides peuvent aussi être utilisées adéquatement pour séparer des suites d'instructions qui réalisent des tâches différentes, par exemple des fonctions différentes.

2.1 Boucle à 5 itérations

Ecrivez un programme en langage d'assemblage MIPS qui effectue une boucle de 5 itérations. Le programme doit être placé dans un fichier nommé `spim-loop-5.s`. Le programme maintient le nombre d'itérations restant à effectuer à l'aide du registre `a0`. Ce registre est décrémenté à chaque itération de la boucle.

▲ Les détails suivants sont importants pour résoudre cet exercice. Premièrement, les noms des registres doivent être préfixés du caractère « \$ ». Par exemple, l'instruction qui charge la valeur 5 dans le registre a0 s'écrit **li \$a0, 5**. Deuxièmement, comme `main` est considéré comme une fonction, il est nécessaire de retourner à l'appelant de `main` une fois la boucle exécutée. Le retour à l'appelant est réalisé avec l'instruction **jr \$ra**.

Exécutez votre programme pas à pas. Vérifiez que votre boucle se termine et le nombre d'itérations effectuées. Observez l'évolution du registre a0 au cours des itérations successives de la boucle.

Q3) Programme en langage d'assemblage MIPS effectuant une boucle à 5 itérations.

L'instruction **li** est une pseudo-instruction. En quelle(s) instruction(s), SPIM l'a-t-il traduite ?

Q4) Traduction de la pseudo-instruction `li $a0, 5`.

Si vous omettez l’instruction `jr` à la fin de votre programme, que se passe-t-il ? Confirmez votre réponse en exécutant une version de votre programme dans laquelle l’instruction `jr` est omise.

Q5) Comportement du programme sans instruction jr.

D'où provient la valeur de `ra` utilisée dans l'instruction `jr $ra` ?

Q6) Comment la valeur de r_a a-t-elle été obtenue ?

Décodez l’instruction de branchement conditionnel utilisée dans votre programme. De ce code d’instruction, dérivez la valeur immédiate `offset`. A quoi correspond cette valeur ? Y a-t-il une différence par rapport à ce qui a été expliqué durant le cours ?

Q7) Décodage de l'instruction de branchement conditionnel.

2.2 Affichage d'un entier

L'objectif de cet exercice est d'utiliser un appel système pour afficher un entier à la console. La façon dont un appel système est réalisé ainsi que la liste des appels système sont décrits en Section A.5. Pour afficher un entier à la console, l'appel système n°1 (aussi appelé `print int`) va être invoqué à l'aide de l'instruction `syscall`. Le numéro de l'appel système doit avoir été placé préalablement dans le registre `v0`. Il en va de même pour la valeur de l'entier à afficher qui doit être placée dans le registre `a0`.

L'extrait de programme montré à la Figure 3 illustre comment afficher l'entier 603 à la console. Cet entier est placé dans le registre `a0` et le numéro d'appel système (1) est placé dans le registre `v0`. L'appel système est invoqué avec l'instruction `syscall`.

```
1      # entier a afficher
2      li      $a0, 603
3      # numero de l'appel systeme
4      li      $v0, 1
5      syscall
```

FIGURE 3 – Invocation de l’appel système “print_int”.

Dans cet exercice, vous devez modifier le programme précédent (boucle à 5 itérations) de sorte qu'à chaque itération la valeur du registre a0 soit affichée à la console. Le programme résultant doit être sauvegardé dans le fichier `spim-loop-5-print.s`. Exécutez ce programme pour en vérifier le bon fonctionnement.

⚠ La console de SPIM est placée dans une fenêtre séparée. Si celle-ci n'est pas visible, il suffit d'aller dans le menu *Window* et de cocher *Console*.

Q8) Programme en langage d'assemblage effectuant 5 itérations et affichant le nombre d'itérations restant à la console.

2.3 Affichage d'une chaîne de caractères

Dans cet exercice, il vous est demandé d'utiliser l'appel système n°4 (`print_string`, Section A.5) afin d'afficher la chaîne de caractères de votre choix à la console. Cet appel système prend un seul argument qui est l'adresse du premier caractère de la chaîne à afficher. Cet argument est passé dans le registre `a0`.

La première étape pour résoudre cet exercice est de définir une chaîne de caractères en mémoire. La Figure 4 illustre la représentation en mémoire de la chaîne de caractères « Hello World of MIPS ». Chaque caractère est représenté par son code ASCII et occupe un octet en mémoire. Par exemple, le caractère 'H' est représenté par le code ASCII 0x48. Les codes des caractères successifs de la chaîne sont placés en mémoire à des adresses consécutives. Ainsi, dans l'exemple, le premier caractère est stocké à l'adresse 0x10010000, le second à l'adresse 0x10010001, etc. ⚠ Ces adresses peuvent être différentes dans votre programme. Un *label* `str` permet de désigner l'adresse du premier caractère de la chaîne, soit 0x10010000.

Il est important de remarquer que la longueur de la chaîne n'est pas stockée en mémoire. En revanche, la fin de la chaîne est indiquée par le code 0. Ce code est visible à la Figure 4 : le dernier caractère de la chaîne, ' S ', est suivi par le code 0. Cette représentation de chaînes de caractères est appelée *chaîne à zéro terminal* (*nul/zero-terminated string*).

str: 0x10010000	'H'
0x10010001	'e'
0x10010002	'l'
0x10010003	'l'
0x10010004	'o'
0x10010005	' '
0x10010011	'P'
0x10010012	'S'
0x10010013	0

FIGURE 4 – Chaîne de caractères à zéro terminal en mémoire.

```

1      .data
2  str:  .asciiz "Hello World of MIPS\n"
3
4      .text
5  main:
6      # suite du programme ...
7      la    $a0, str
8      li    $v0, 4
9      syscall

```

FIGURE 5 – Définition d'une chaîne de caractères, obtention de son adresse et invocation de l'appel système `print_string(4)`.

Définir une telle chaîne en mémoire peut être réalisé à l'aide de directives du langage d'assemblage (voir Section A.2), comme montré à la Figure 5. La directive `.ascii` crée une chaîne à zéro terminal en mémoire, sur base d'un littéral chaîne de caractères entouré de guillemets ("). La chaîne est définie dans le segment de données du programme, grâce à la directive `.data`. Ce qui est déclaré dans le segment de données est placé en mémoire RAM. De même, la directive `.text` indique que ce qui suit est placé dans le segment d'instructions (et est typiquement placé en mémoire ROM/Flash). Afin de pouvoir faire référence à la chaîne de caractères, elle est précédée d'un label (`str`). L'adresse mémoire de la chaîne de caractères peut ainsi être récupérée et copiée dans un registre à l'aide de la pseudo-instruction ***la*** (*Load Address*).

Nommez votre programme `spim-print-str.s` et vérifiez son bon fonctionnement en le chargeant et l'exécutant à l'aide de SPIM.

Q9) Programme affichant une chaîne de caractères à la console.

Afin de vérifier votre compréhension de l'encodage d'une chaîne de caractères en mémoire, veuillez donner la valeur du mot de 32 bits stocké à l'adresse désignée par le *label* `str`. Afin d'obtenir l'adresse à laquelle correspond le *label* `str`, vous pouvez exécuter votre programme jusqu'à passer la pseudo-instruction `la $a0, str`. L'adresse sera alors chargée dans le registre `a0`. Une autre possibilité consiste à définir `str` comme un symbole global (voir directive `.globl` en Section A.2) et lister les symboles avec le menu *Simulator / Display Symbols*.

Allez ensuite voir dans la zone de mémoire de données via le panneau central du simulateur, onglet *Data*. Dans ce panneau, chaque ligne donne le contenu de 16 octets consécutifs en mémoire. L'adresse du premier de ces octets est en début de ligne.

Notez ci-dessous, en hexadécimal, l'adresse correspondant à `str` et le mot de 32 bits situé à cette adresse. Comment interprétez-vous la valeur de ce mot ?

Q10) Adresse et valeur du mot de 32 bits situé à l'adresse `str`, en hexadécimal.

.....

.....

2.4 Somme de deux entiers

Dans cet exercice, il vous est demandé d'écrire un programme qui demande deux entiers à l'utilisateur et qui en affiche la somme. Pour demander un entier à l'utilisateur, utilisez l'appel système n°5 (`read int`). Celui-ci retourne l'entier lu à la console dans le registre `v0`. **▲** Si la chaîne ne correspond pas à un entier, l'appel système retourne 0.

Nommez votre programme `spim-add-int.s` et testez-le programme avec plusieurs couples d'entiers.

Q11) Programme calculant la somme de deux entiers lus à la console.

2.5 Lecture d'entiers et tests de conditions

Dans cet exercice, il vous est demandé de produire un programme qui demande, en boucle, un entier à l'utilisateur. Si cet entier est inférieur à 0, la boucle se termine. Si l'entier est strictement supérieur à 10, la chaîne de caractères « trop grand » est affichée à la console. Sinon, la chaîne de caractères « valeur acceptée » est affichée à la console.

▲ Ce programme commence à devenir plus complexe. Il devrait s'étaler sur environ 30-40 lignes. Assurez-vous de bien le structurer, en employant l'indentation à bon escient, en insérant des lignes vides pour séparer les différentes tâches et en ajoutant éventuellement des commentaires (en utilisant le caractère spécial `#`).

- Pour demander un entier à l'utilisateur, utilisez l'appel système n°5 comme dans l'exercice 2.4.
- Pour adapter le comportement du programme aux entiers fournis par l'utilisateur, il faut réaliser des comparaisons et l'équivalent de `if/else`, à l'aide d'instructions de branchement conditionnel telles que `beq` (*branch on equal*), `bne` (*branch on not equal*), `blt` (*branch on less than*), etc.
- Pour afficher les chaînes de caractères, utilisez l'appel système n°4 comme dans l'exercice 2.3.

Nommez votre programme `spim-read-int.s` et vérifiez-en le bon fonctionnement en le chargeant et l'exécutant à l'aide de SPIM. Fournissez des valeurs différentes permettant de tester toutes les conditions du programme.

Q12) Programme lisant des entiers à la console.

2.6 Conversion en binaire

Dans cet exercice, il vous est demandé d'écrire un programme qui demande un entier à l'utilisateur et qui affiche cet entier en binaire. Utilisez à cet effet la méthode effectuant des divisions euclidiennes successives, vue au Chapitre 2 du cours. Pour résoudre cet exercice, vous aurez besoin de l'instruction `divu x, y`. Pour rappel, cette instruction calcule le quotient et le reste de la division de x par y (en considérant que ces nombres sont non-signés). Le quotient est placé dans le registre `LO` tandis que le reste est placé dans le registre `HI`. Votre algorithme peut produire les bits de la représentation binaire en commençant par le bit de poids le plus faible.

Placez le programme résultant dans un fichier nommé `spim-to-binary.s`. Testez ce programme avec plusieurs nombres naturels.

Q13) Programme convertissant un naturel en binaire.

3 Exercices sur les appels de fonctions

3.1 Mécanisme d'appel de fonction

Cet exercice vise à bien comprendre le mécanisme d'appel de fonction. Il implique l'usage des instructions `jal` et `jr` ainsi que du registre `ra` pour conserver l'adresse de retour. ⚠ Assurez-vous de bien les comprendre !

Il vous est demandé d'effectuer un appel à une fonction appelée `fct` qui ne prend pas d'argument et qui ne retourne pas de résultat. Cette fonction affiche un message pré-déterminé à la console et retourne ensuite à l'appelant. Votre programme doit donc définir deux fonctions : la première correspond au programme principal et est désignée par le label `main`, tandis que la deuxième est la fonction à appeler et est désignée par le label `fct`.

⚠ Il y a en réalité deux appels de fonctions imbriqués dans cet exercice : appel de `main` réalisé par SPIM et appel de `fct` réalisé par `main`. Ceci implique qu'il faille sauvegarder correctement l'adresse de retour. Dans un premier temps, il vous est proposé d'utiliser un autre registre à cet effet plutôt que d'utiliser la pile.

Placez votre programme dans un fichier nommé `spim-function-call.s`. Vérifiez le comportement de votre programme en l'exécutant pas-à-pas. Observez, en particulier, l'évolution du registre *Program Counter* (`pc`) et le registre *Return Address* (`ra`). Exécutez à cet effet votre programme *pas-à-pas*.

Q14) Programme effectuant un appel de fonction.

3.2 Sauvegarde de l'adresse de retour sur la pile

Cet exercice est une variante du précédent dans lequel il s'agit maintenant de sauvegarder l'adresse de retour sur la pile plutôt que dans un registre. Seule la fonction `main` doit effectuer cette sauvegarde car elle est la seule dans cet exemple à effectuer l'appel d'une autre fonction. Avant d'effectuer l'appel, il faut pousser le contenu du registre `ra` sur la pile (*push*). Une fois l'appel effectué, il faut récupérer le contenu de `ra` à partir du sommet de la pile (*pop*). Le sommet de la pile est stocké dans le registre `sp` (*stack pointer*). Les opérations *push* et *pop* sont réalisées sous la forme de suites d'instructions déplaçant le sommet de la pile et effectuant des copies vers et depuis la mémoire (*load* et *store*).

⚠ Attention, une fonction doit toujours rétablir le pointeur de pile dans l'état où elle l'a trouvé.

Q15) Programme effectuant un appel de fonction (sauvegarde de `ra` sur la pile).

3.3 Passage d'arguments et résultat

L'objectif de cette question est de réaliser un appel de fonction dans lequel des paramètres sont passés en arguments et un résultat est retourné. La fonction à implémenter est nommée `mul6`. Elle prend 6 entiers en arguments, en effectue la multiplication et retourne le résultat. Il vous est demandé de suivre la convention MIPS pour le passage des paramètres : les 4 premiers entiers sont passés dans les registres `a0` à `a3` et les 2 arguments excédentaires sont passés sur la pile. Le résultat de la multiplication doit être retourné dans le registre `v0`.

Placez votre programme dans le fichier `spim-mul6.s`. Ce programme doit contenir l'implémentation de la fonction `mul6` ainsi que le programme principal (fonction `main`) effectuant un appel à `mul6` avec des paramètres que vous aurez choisis. Vérifiez le résultat retourné par la fonction.

Q16) Programme effectuant un appel de fonction (passage d'arguments).

3.4 Manipulation de chaînes de caractères

L'objectif de cet exercice est d'implémenter deux fonctions utiles lors de la manipulation de chaînes de caractères. Les chaînes de caractères sont de type « à zéro terminal ». Les fonctions à implémenter sont les suivantes :

- `strlen` : détermine la longueur d'une chaîne à zéro terminal. La fonction prend un seul argument : l'adresse du premier caractère de la chaîne de caractères, passé dans le registre `a0`. La fonction retourne dans le registre `v0` la longueur de la chaîne. Note : une solution à cet exercice a été discutée durant la partie théorique du cours.
- `strcmp` : compare deux chaînes de caractères selon l'ordre lexicographique. La fonction prend 2 arguments : les adresses `s1` et `s2` de deux chaînes de caractères, passées dans les registres `a0` et `a1` respectivement. La fonction retourne dans le registre `v0` la valeur 0 si les deux chaînes sont identiques ; la valeur -1 si la chaîne `s1` est inférieure à la chaîne `s2` ; la valeur +1 sinon.

L'implémentation des deux fonctions nécessite de lire une partie ou la totalité des caractères des chaînes. L'instruction `lb` (*Load Byte*) permet de charger dans un registre l'octet situé à une adresse donnée.

Placez vos fonctions ainsi qu'un programme de test dans le fichier `spim-strlen-strcmp.s`. Assurez-vous de tester vos fonctions sur plusieurs exemples permettant de vérifier les différents cas de figure possibles.

Q17) Fonction `strlen`.**Q18) Fonction `strcmp`.**

3.5 Multiplication... récursive

Dans cet exercice, il s'agit d'effectuer la multiplication de deux nombres naturels de manière récursive. La fonction `mult_rec` prend en arguments deux entiers positifs ou nuls x et y placés dans les registres `a0` et `a1` respectivement. Le produit de x et y sera retourné dans le registre `v0` et affiché à la console.

Il est interdit d'utiliser les instructions de multiplication telles que `mult`. La fonction doit être implémentée récursivement, c'est-à-dire que la fonction doit s'appeler elle-même un certain nombre de fois. Remarquez à cet effet que $x \times y$ peut être obtenu comme suit

$$p = \begin{cases} y + (x - 1) \times y & \text{si } x > 0 \\ 0 & \text{sinon} \end{cases}$$

Posez-vous également la question « y a-t-il un avantage à effectuer la récursion sur x ou sur y ? »

Placez votre fonction `mult_rec` ainsi qu'un programme de test dans le fichier `spim-mult-rec.s`. Observez l'exécution de votre programme *pas-à-pas*. Prêtez particulièrement attention aux registres *Program Counter* (`pc`), *Return Address* (`ra`) et au *Stack Pointer* (`sp`) lors des appels récursifs.

Q19) Fonction `mult_rec`.

4 Exercices sur les tableaux

4.1 Recherche dans un tableau

Cet exercice vise à rechercher un entier dans un tableau d'entiers triés. A cet effet, votre programme doit déclarer un tableau trié d'entiers en mémoire. Utilisez pour cela la directive `.word` comme illustré ci-dessous. Dans l'exemple, un tableau de 5 entiers de 32 bits est créé et ses cellules sont initialisées avec les valeurs -17, 5, 9, 123 et 1024. L'adresse de la première cellule du tableau est désignée par le *label* `tableau`.

```
1 | tableau:
2 |     .word -17, 5, 9, 123, 1024
```

Le programme doit également contenir la définition d'une fonction `find_int`. Cette fonction prend comme premier argument l'adresse de la première cellule d'un tableau, comme second argument la taille du tableau et comme troisième argument l'entier recherché. La fonction retourne la position dans le tableau de l'entier recherché si celui-ci s'y trouve. La fonction retourne -1 si l'entier ne se trouve pas dans le tableau. Vous pouvez bien entendu tirer parti du fait que le tableau est trié, de façon à en diminuer la complexité algorithmique.

Placez votre fonction `find_int`, la déclaration d'un tableau trié d'entiers ainsi qu'un programme de test dans le fichier `spim-find-int.s`. Assurez-vous de rechercher des entiers qui sont ou pas dans le tableau, en positions début, fin et milieu, de façon à tester les différents cas auxquels votre implémentation sera exposée.

Q20) Fonction `find_int`.

4.2 Structure de contrôle `switch`

Le chapitre 4 du cours a montré comment une structure de contrôle `switch` peut être exprimée en langage d'assemblage, en utilisant une séquence d'instructions de test avec la variable de décision. Dans cette approche, il y a une instruction de branchement conditionnel par cas du `switch`. Cette approche est illustrée à la Figure 6.

```
1 | switch (a) {
2 | case 5:
3 |     /* code du cas 5 */
4 |     break;
5 | case 13:
6 |     /* code du cas 13 */
7 |     break;
8 | case 37:
9 |     /* code du cas 37 */
10 |    break;
11 | default:
12 |     /* code autre */
13 | }

1 |
2 | case5:
3 |     li $t0, 5
4 |     bne $a0, $t0, case13
5 |     # code du cas 5
6 |     j end_switch
7 |
8 | case13:
9 |     li $t0, 13
10 |    bne $a0, $t0, case37
11 |    # code du cas 13
12 |    j end_switch
13 |
14 | case37:
15 |     li $t0, 37
16 |     bne $a0, $t0, case_default
17 |     # code du cas 37
18 |     j end_switch
19 |
20 | case_default:
21 |     # code autre
22 |
23 | end_switch:
```

FIGURE 6 – Implémentation possible d'une structure `switch` en langage d'assemblage MIPS.

Si les cas sont consécutifs, il est possible d'utiliser une autre approche plus économe en terme d'instructions : l'utilisation d'une table de branchement. Un exemple d'un `switch` qui se prête à une telle implémentation est illustré à la Figure 7. Une table de branchement contient les adresses auxquelles il faut brancher pour chacun des cas. L'index dans la table est obtenue à partir de la variable de décision (`a` dans l'exemple). La Figure 7 illustre une table de branchement composée de 4 adresses correspondant aux cas 5 à 8 (via les labels `case5` à `case8`). La directive `.word` permet de définir un ou plusieurs mode de 32 bits en mémoire. Pour effectuer le branchement, il faut d'abord déterminer l'index dans la table. Dans le cas de l'exemple, cet index sera `a - 5`. Cet index est converti en adresse mémoire,

de façon à aller lire l'adresse de branchement correspondante dans la table, à l'aide de l'instruction `lw` (*Load Word*). Dans l'exemple, cette adresse a la forme `tab + 4 × (a - 5)`. Il suffit ensuite de brancher à l'adresse lue dans la table, à l'aide de l'instruction `jr` (*Jump Register*).

```

1 switch (a) {
2 case 5:
3     /* code du cas 5 */
4     break;
5 case 6:
6     /* code du cas 6 */
7     break;
8 case 7:
9     /* code du cas 7 */
10    break;
11 case 8:
12    /* code du cas 8 */
13    break;
14 default:
15    /* code autre */
16 }

```

```

1 .data
2 tab: .word case5, case6, case7, case8
3
4 .text
5 main:
6     # chercher adresse branchement
7     # dans table
8     lw ...
9     # brancher vers adresse obtenue,
10    jr ...
11 case5:
12
13 case6:
14
15 case7:
16
17 case8:
18
19 end_switch:
20     jr $ra

```

FIGURE 7 – Structure `switch` avec cas consécutifs.

Placez le programme résultant dans un fichier nommé `switch-table.s`. Testez ce programme avec plusieurs valeurs de `a` de façon à vérifier le bon fonctionnement de votre implémentation.

Q21) Programme implémentant une structure `switch` avec une table de branchement.

4.3 Conversion en hexadécimal

Adaptez le programme de la Section 2.6 de façon à effectuer la conversion en hexadécimal. Deux adaptations sont nécessaires. La première est facile : il faut changer le diviseur utilisé dans l'algorithme. La seconde est plus complexe : il faut pouvoir afficher des symboles hexadécimaux à la console, en fonction des restes produits par l'algorithme.

Supposons que vous disposiez de 16 chaînes de caractères, correspondant à chacun des symboles hexadécimaux, telles qu'illustrées à la Figure 8. Ces chaînes sont consécutives en mémoire. Par exemple, l'adresse de "0" est `hex`, alors que "1" est `hex+2`, "2" est `hex+4`, etc. Il est donc possible de calculer l'adresse de chaque chaîne en fonction de l'adresse de la première ("0") et d'un décalage proportionnel à la valeur du symbole à afficher... Le décalage est un multiple de 2 car chaque chaîne est composée du caractère hexadécimal et du caractère 0 signalant la fin de chaîne.

```

1 .data
2 hex: .asciiz "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "A", "B", "C", "D", "E", "F"

```

FIGURE 8 – Définition de 16 chaînes consécutives en mémoire, chacune correspondant à un symbole hexadécimal.

Une autre possibilité serait de définir une chaîne contenant les 16 caractères hexadécimaux. Le caractère voulu peut alors être lu en utilisant l'adresse de base de la chaîne plus un décalage correspondant au symbole à afficher. Pour aller lire un caractère de la chaîne, l'instruction de lecture en mémoire `lb` (*Load Byte*) peut être utilisée. Celle-ci fonctionne exactement de la même manière que `lw` (*Load Word*) à l'exception qu'elle ne lit qu'un octet plutôt qu'un mot de 32 bits.

Placez le programme résultant dans un fichier nommé `spim-to-hex.s`. Testez ce programme avec plusieurs nombres naturels.

Q22) Programme convertissant un naturel en hexadécimal.

```
1 | .data  
2 | hex: .asciiz "0123456789ABCDEF"
```

FIGURE 9 – Définition d'une chaîne de 16 symboles hexadécimaux.

5 Idées d'exercices supplémentaires

5.1 Nombre variable d'arguments

Implémentez une fonction qui accepte un nombre variables d'entiers en arguments. Le premier argument (passé dans `$a0`) est obligatoirement présent et représente le nombre d'arguments qui suivent. Les arguments suivants sont passés sur la pile. La fonction calcule la moyenne de ces entiers et retourne la moyenne dans `v0`.

5.2 Tableaux d'entiers

Une alternative à l'exercice précédent consiste à passer à la fonction un tableau d'entiers. Le premier argument de la fonction (`$a0`) est l'adresse de la première cellule du tableau, tandis que le second est le nombre d'éléments du tableau (`$a1`).

5.3 Afficher une chaîne de caractères

Supposons que vous ne disposiez pas de l'appel système 4 (`print int`). Implémentez une fonction `strprint` qui prend en entrée une chaîne de caractères à zéro terminal et qui l'affiche caractère par caractère, en utilisant l'appel système 11 (`print_char`).

5.4 Factorielle

Fournissez deux implémentations de la fonction factorielle. La première doit être itérative et la seconde doit être récursive (exercice fait en cours).

5.5 Attaque par buffer overflow

Attaquez le programme suivant par buffer overflow. Ce programme demande à l'utilisateur une chaîne de caractères. La chaîne est stockée sur la pile, où 11 octets sont prévus, ce qui permet d'accueillir une chaîne de 10 caractères, suivie d'un zéro terminal. Malheureusement, le programme permet que jusqu'à 16 caractères soient entrés...

```

1  .data
2  msgi: .asciiz "Input:"
3  msggh: .asciiz "You've been hacked.\n"
4
5  .text
6  main:
7      addiu $sp, $sp, -4
8      sw     $ra, 0($sp)
9      jal    ask_user
10     lw     $ra, 0($sp)
11     addiu $sp, $sp, 4
12     jr     $ra
13
14  ask_user:
15     addiu $sp, $sp, -11
16     li     $v0, 4 # print string
17     la     $a0, msgi
18     syscall
19     li     $v0, 8 # read string
20     move   $a0, $sp
21     li     $a1, 16
22     syscall
23     addiu $sp, $sp, 11
24     jr     $ra
25
26  .text 0x00402020
27  super_power():
28     la     $a0, msggh
29     li     $v0, 4 # print string
30     syscall
31     li     $v0, 10 # exit
32     syscall
33     jr     $ra

```

Votre objectif, en exploitant la vulnérabilité du programme est que la fonction `super_power()` soit appelée, alors qu'il n'y a aucun appel de celle-ci dans le programme. Lorsque cette fonction est appelée, le message « *You've been hacked.* » est imprimé à la console. **⚠ Attention, vous ne pouvez pas modifier le code du programme ! Votre seul moyen d'agir est de trouver la bonne chaîne de caractères à lui donner.** Afin de bien comprendre comment effectuer l'attaque du programme, assurez-vous de comprendre ce qui se situe sur le sommet de la pile.

5.6 Combinaisons

Implémentez une fonction calculant de façon récursive le nombre de façons de prendre un sous-ensemble de taille k parmi un ensemble de taille n . Implémentez éventuellement une version mettant en oeuvre la mémorisation (triangle de Pascal).

6 Exercices sur les entrées/sorties

Les exercices proposés dans cette section sont optionnels. Ils requièrent des concepts qui n'ont pas nécessairement été vus lors de la partie théorique du cours, tels que les « entrées-sorties mappées en mémoire » et les interruptions.

6.1 Entrées-sorties mappées en mémoire

Dans cet exercice, il s'agit d'utiliser des entrées-sorties dites « mappées en mémoire » (*memory-mapped input/output*). Pour rappel, ce type d'entrées-sorties repose sur l'accès à des registres de périphériques ou contrôleurs d'entrées-sorties **comme s'il s'agissait de cellules mémoires**, c'est-à-dire en utilisant des instructions de type `lw` et `sw` dans l'architecture MIPS.

Le simulateur SPIM définit 4 registres de ce type qui permettent d'une part de lire les caractères entrés au clavier (périphérique d'entrée) et d'autre part d'envoyer des caractères vers la console (périphérique de sortie). Ces registres sont résumés à la Table 2. **Attention ! Pour que les entrées-sorties « mappées en mémoire » fonctionnent sous SPIM, il est nécessaire d'activer cette option via le menu “Préférences”, sous l'onglet “MIPS”, cocher “Enable Mapped IO”.**

Nom	Adresse en mémoire	Description
KBD_RX_CTRL	0xffff0000	Le bit 0 de ce registre est positionné à 1 lorsqu'un caractère est disponible. Le bit 1 autorise le déclenchement d'interruptions lors de la disponibilité de caractères. Les autres bits ne sont pas utilisés.
KBD_RX_DATA	0xffff0004	Les 8 bits de poids faible de ce registre contiennent le code d'un caractère reçu du clavier. Les autres bits valent 0.
DISP_TX_CTRL	0xffff0008	Le bit 0 de ce registre est positionné à 1 lorsqu'un caractère peut être envoyé vers la console. Le bit 1 autorise le déclenchement d'interruptions lorsque le périphérique de sortie est prêt. Les autres bits ne sont pas utilisés.
DISP_TX_DATA	0xffff000c	Les 8 bits de poids faible de ce registre contiennent le code d'un caractère à envoyer vers la console. Les autres bits ne sont pas utilisés.

TABLE 2 – Registres mappés en mémoire supportés par SPIM.

L'objectif de cet exercice est de lire les caractères au clavier et d'afficher leur code à l'aide de l'appel système `syscall`. Placez votre programme dans un fichier nommé `spim-mmio.s`.

Q23) Lecture de caractères par entrées-sorties « mappées en mémoire ».

6.2 Utilisation d'un timer

Un *timer* (chronomètre) est un périphérique permettant de mesurer le temps qui s'écoule. Il peut par exemple être utilisé pour effectuer une action à intervalle régulier, comme dans cet exercice. Le simulateur SPIM supporte un périphérique *timer*. **Attention toutefois, ce timer s'écoule uniquement lorsque le programme est en mode « run ». Il ne progresse pas en mode « single step ».**

Plusieurs registres permettent de contrôler le *timer*. Le registre `Count` est incrémenté à intervalle régulier (toutes les 10 millisecondes dans SPIM). Le registre `Compare` est comparé au registre `Count` et dès que les deux sont égaux, cette condition est signalée au processeur : le bit 15 du registre `Cause` est mis à 1, ce qui signale une interruption matérielle. Ce bit doit être remis à zéro par le logiciel.

Les registres `Cause`, `Count` et `Compare` sont des registres spéciaux qui font partie du coprocesseur 0 (CP0 pour les intimes). On y accède à l'aide des instructions `mfc0` (*move from CP0*) et `mtc0` (*move to CP0*). L'extrait de code suivant montre comment placer la valeur 100 dans le registre `Compare` de CP0 à l'aide de l'instruction `mtc0`. Les index des registres de CP0 sont repris à la Table 3.

```
1 | li    $a0, 100
2 | mtc0 $a0, $11
```

L'objectif de cet exercice est d'afficher un message à la console toutes les 5 secondes. Pour cela, la valeur 500 doit être chargée dans le registre `Compare` et le registre `Count` doit être remis à 0. Le programme doit alors surveiller le bit 15 du registre `Cause`. Lorsqu'il passe à 1, cela signale l'expiration du *timer* et le message doit être affiché à la console. Le bit 15 de `Cause` doit alors être remis à 0, ainsi que le registre `Count`.

Registre	Index
Count	9
Compare	11
Cause	13

TABLE 3 – Registres du coprocesseur 0.

Placez votre programme dans le fichier `spim-timer.s`. Lancez-en l'exécution et vérifiez que votre message est bien affiché à la console toutes les 5 secondes.

Q24) Affichage d'un message à intervalle régulier à l'aide d'un *timer*.

A Annexes

A.1 Langage d'assemblage

SPIM supporte directement le langage d'assemblage MIPS. Il se charge de convertir un programme en langage d'assemblage vers le langage machine et de charger les données et instructions définies dans le programme vers les zones mémoire correspondantes. Les particularités du langage d'assemblage supporté par le simulateur sont résumées ci-dessous.

- Chaque instruction occupe une ligne : le mnémonique de l'instruction vient en premier, suivi des opérandes. Le registre destination est généralement le premier opérande.
- Des commentaires peuvent être écrits à la suite du caractère `#`. Le commentaire s'étend jusqu'à la fin de la ligne.
- Les noms des registres doivent être préfixés par le caractère `$`.
- Le point d'entrée du programme doit être désigné par le *label* `main`. La fin du programme se traduit par un retour vers l'appelant de `main`, en utilisant `jx $ra`.
- Plusieurs pseudo-instructions sont disponibles telles que
 - `li` (*load immediate*) : charge une constante dans un registre
 - `la` (*load address*) : charge une adresse dans un registre, typiquement à partir d'un *label*.
- Plusieurs directives sont supportées, voir Section A.2
- Plusieurs appels systèmes sont supportés, voir Section A.5

⚠ Le simulateur est parfois assez permissif et se permet de traduire automatiquement (et silencieusement) certaines instructions qui ne devraient pas être permises. Cela pourrait causer de la confusion de votre part. Par exemple,

- L'instruction `add $a0, $a1, 5` ne devrait pas être permise car le troisième opérande devrait être un registre et non une constante. Pourtant, SPIM va la traduire automatiquement en `addi $a0, $a1, 5`.
- L'instruction `jx loop` sera traduite en `j loop`, mais il est important de comprendre que `jx` (*jump register*) devrait normalement prendre un registre comme opérande !

A.2 Directives de l'assembleur

Le langage d'assemblage MIPS supporte plusieurs directives permettant par exemple de définir des chaînes de caractères en mémoire ou des tableaux d'entiers. La Table 4 donne une brève description des directives qui pourraient être utiles dans ce TP.

TABLE 4 – Liste de directives utiles de l’assembleur MIPS.

Directive	Description
<code>.data [addr]</code>	Indique que les déclarations suivantes (données et instructions) sont mises dans un segment <u>mémoire de données</u> . Il est possible d’imposer l’adresse mémoire à laquelle le segment commence, en utilisant l’argument optionnel <i>addr</i> .
<code>.text [addr]</code>	Indique que les déclarations suivantes (données et instructions) sont mises dans un segment <u>mémoire d’instructions</u> . Il est possible d’imposer l’adresse mémoire à laquelle le segment commence, en utilisant l’argument optionnel <i>addr</i> .
<code>.ascii str</code>	Stocke le littéral chaîne de caractères <i>str</i> en mémoire selon le codage ASCII et sans zéro terminal.
<code>.asciiz str</code>	Stocke le littéral chaîne de caractères <i>str</i> en mémoire selon le codage ASCII et avec zéro terminal.
<code>.byte b₁, ..., b_n</code>	Stocke les <i>n</i> littéraux entiers <i>b₁, ..., b_n</i> sous forme de mots de 8 bits à des emplacements consécutifs en mémoire.
<code>.half s₁, ..., s_n</code>	Stocke les <i>n</i> littéraux entiers <i>s₁, ..., s_n</i> sous forme de mots de 16 bits à des emplacements consécutifs en mémoire.
<code>.word w₁, ..., w_n</code>	Stocke les <i>n</i> littéraux entiers <i>w₁, ..., w_n</i> sous forme de mots de 32 bits à des emplacements consécutifs en mémoire.
<code>.space n</code>	Réserve une zone de <i>n</i> octets.
<code>.float f₁, ..., f_n</code>	Stocke les <i>n</i> littéraux flottants <i>f₁, ..., f_n</i> sous forme de mots de 32 bits au format IEEE 754 simple précision à des emplacements consécutifs en mémoire.
<code>.double d₁, ..., d_n</code>	Stocke les <i>n</i> littéraux flottants <i>d₁, ..., d_n</i> sous forme de mots de 64 bits au format IEEE 754 double précision à des emplacements consécutifs en mémoire.
<code>.align n</code>	Force l’alignement des données suivantes à une adresse qui est un multiple de 2^n .
<code>.extern sym size</code>	Déclare le symbole <i>sym</i> comme étant global et de taille <i>size</i> . Cela peut être utilisé pour déclarer un symbole défini ailleurs, p.ex. dans un autre fichier.
<code>.globl sym</code>	Déclare le symbole <i>sym</i> comme étant global. Cela permet de référencer ce symbole à partir d’autres fichiers. Dans le simulateur SPIM, cela permet aussi que ce symbole soit listé via le menu <i>Simulator / Display Symbols</i> .
<code>.kdata [addr]</code>	Similaire à <code>.data</code> mais sera placé dans une zone mémoire réservée au système.
<code>.ktext [addr]</code>	Similaire à <code>.text</code> mais sera placé dans une zone mémoire réservée au système.

TABLE 5 – Liste des registres GPR MIPS.

Nom	Numéro	Description
zero	0	Valeur constante 0
v0, v1	2, 3	Retour de valeur par une fonction
a0..a3	4..7	Temporaires (passage d’arguments)
t0..t7	8..15	Temporaires, préservés par l’appelant (<i>caller-saved</i>)
s0..s7	16..23	Temporaires, préservés par l’appelé (<i>callee-saved</i>)
t8, t9	24, 25	Temporaires, préservés par l’appelant (<i>caller-saved</i>)
k0, k1	26, 27	Réservé pour le noyau (<i>kernel</i>)
gp	28	Adresse de l’espace de données global
sp	29	Adresse du sommet de la pile (<i>stack pointer</i>)
fp	30	Adresse du cadre de pile (<i>frame pointer</i>)
ra	31	Adresse de retour lors d’un appel de fonction (<i>return address</i>)

A.3 Architecture MIPS

Cette section rappelle les éléments les plus importants de l’architecture et du jeu d’instructions MIPS. La Table 5 fournit la liste des registres d’usage général (GPR) utilisables avec MIPS ainsi que les conventions d’usage. Chaque registre stocke un mot de 32 bits. En plus de ces registres, l’architecture définit les registres HI et LO utilisés lors des multiplications et divisions.

La Table 6 contient une liste d’instructions MIPS et leur description. La définition du fonctionnement d’une instruction peut utiliser des opérations spéciales telles que *s-ext* qui indique une extension signée, *0-ext* qui indique une extension non-signée, ou encore *address* qui indique que l’on prend l’adresse d’un label. Les instructions de branchement conditionnel font référence à l’opération *branch* définie comme $PC \leftarrow PC + 4 + s\text{-ext}(\text{offset} \parallel 0^2)$. Le mnémonique des pseudo-instructions est indiqué en italique (p.ex. *LI*). Les accès mémoire

sont effectués par octet (B), par mot de 16 bits (H) ou par mot de 32 bits (W) et ont lieu à des adresses alignées sur la taille de ces mots.

TABLE 6 – Instructions MIPS.

Mnémonique	Opérandes	Description	Fonctionnement
ADD	<i>rd, rs, rt</i>	Addition	$\text{GPR}[rd] \leftarrow \text{GPR}[rs] + \text{GPR}[rt]$
ADDI	<i>rt, rs, imm</i>	Addition immédiate	$\text{GPR}[rt] \leftarrow \text{GPR}[rs] + \text{s-ext}(imm)$
ADDU	<i>rd, rs, rt</i>	Addition non signée (pas d'exception)	$\text{GPR}[rd] \leftarrow \text{GPR}[rs] + \text{GPR}[rt]$
ADDIU	<i>rt, rs, imm</i>	Addition immédiate non signée (pas d'exception)	$\text{GPR}[rt] \leftarrow \text{GPR}[rs] + \text{0-ext}(imm)$
AND	<i>rd, rs, rt</i>	ET logique bit à bit	$\text{GPR}[rd] \leftarrow \text{GPR}[rs] \text{ AND } \text{GPR}[rt]$
ANDI	<i>rt, rs, imm</i>	ET logique immédiat bit à bit	$\text{GPR}[rt] \leftarrow \text{GPR}[rs] \text{ AND } \text{0-ext}(imm)$
BEQ	<i>rs, rt, offset</i>	Branchement si égalité	if $\text{GPR}[rs] = \text{GPR}[rt]$ then <i>branch</i>
BNE	<i>rs, rt, offset</i>	Branchement si différent	if $\text{GPR}[rs] \neq \text{GPR}[rt]$ then <i>branch</i>
BGEZ	<i>rs, offset</i>	Branchement si ≥ 0	if $\text{GPR}[rs] \geq 0$ then <i>branch</i>
BGTZ	<i>rs, offset</i>	Branchement si > 0	if $\text{GPR}[rs] > 0$ then <i>branch</i>
BLEZ	<i>rs, offset</i>	Branchement si ≤ 0	if $\text{GPR}[rs] \leq 0$ then <i>branch</i>
BLTZ	<i>rs, offset</i>	Branchement si < 0	if $\text{GPR}[rs] < 0$ then <i>branch</i>
BGE	<i>r1, r2, offset</i>	Branchement si $\geq$	if $\text{GPR}[r1] \geq \text{GPR}[r2]$ then <i>branch</i>
BGT	<i>r1, r2, offset</i>	Branchement si $>$	if $\text{GPR}[r1] > \text{GPR}[r2]$ then <i>branch</i>
BLE	<i>r1, r2, offset</i>	Branchement si $\leq$	if $\text{GPR}[r1] \leq \text{GPR}[r2]$ then <i>branch</i>
BLT	<i>r1, r2, offset</i>	Branchement si $<$	if $\text{GPR}[r1] < \text{GPR}[r2]$ then <i>branch</i>
DIV	<i>rs, rt</i>	Division	$(\text{HI}, \text{LO}) \leftarrow \text{GPR}[rs] / \text{GPR}[rt]$ HI = reste ; LO = quotient
J	<i>target</i>	Saut	$\text{PC} \leftarrow \text{PC}_{31..28} \parallel \text{target} \parallel 0^2$
JAL	<i>target</i>	Appel de fonction	$\text{GPR}[31] \leftarrow \text{PC} + 4$ $\text{PC} \leftarrow \text{PC}_{31..28} \parallel \text{target} \parallel 0^2$
JALR	<i>rs</i>	Appel de fonction (indirect)	$\text{GPR}[31] \leftarrow \text{PC} + 4$ $\text{PC} \leftarrow \text{GPR}[rs]$
JR	<i>rs</i>	Saut (indirect)	$\text{PC} \leftarrow \text{GPR}[rs]$
LA	<i>reg, label</i>	Charge une adresse	$\text{GPR}[reg] \leftarrow \text{address}(label)$
LI	<i>reg, imm</i>	Charge une constante	$\text{GPR}[reg] \leftarrow imm$
LB, LH, LW	<i>rt, offset(base)</i>	Lecture mémoire ; B : <i>byte</i> (1 octet), H : <i>half-word</i> (2 octets) ; W : <i>word</i> (4 octets)	$\text{GPR}[rt] \leftarrow \text{MEM}[\text{GPR}[base] + offset]$
MFHI	<i>rd</i>	Copie du registre <i>hi</i>	$\text{GPR}[rd] \leftarrow \text{HI}$
MFLO	<i>rd</i>	Copie du registre <i>lo</i>	$\text{GPR}[rd] \leftarrow \text{LO}$
MOVE	<i>r1, r2</i>	Copie de registre	$\text{GPR}[r1] \leftarrow \text{GPR}[r2]$
MULT	<i>rs, rt</i>	Multiplication	$(\text{HI}, \text{LO}) \leftarrow \text{GPR}[rs] \times \text{GPR}[rt]$
OR	<i>rd, rs, rt</i>	OU logique bit à bit	$\text{GPR}[rd] \leftarrow \text{GPR}[rs] \text{ OR } \text{GPR}[rt]$
ORI	<i>rt, rs, imm</i>	OU logique immédiat bit à bit	$\text{GPR}[rt] \leftarrow \text{GPR}[rs] \text{ OR } \text{0-ext}(imm)$
SLL	<i>rd, rt, sa</i>	Décalage vers la gauche	$\text{GPR}[rd] \leftarrow \text{GPR}[rt] \ll sa$
SRA	<i>rd, rt, sa</i>	Décalage vers la droite (arithmétique)	$\text{GPR}[rd] \leftarrow \text{GPR}[rt] \gg sa$
SRL	<i>rd, rt, sa</i>	Décalage vers la droite	$\text{GPR}[rd] \leftarrow \text{GPR}[rt] \gg sa$
SLT	<i>rd, rs, rt</i>	Mettre à 1 si inférieur	$\text{GPR}[rd] \leftarrow \text{GPR}[rs] < \text{GPR}[rt]$
SLTU	<i>rd, rs, rt</i>	Mettre à 1 si inférieur (non-signé)	$\text{GPR}[rd] \leftarrow \text{GPR}[rs] < \text{GPR}[rt]$
SUB	<i>rd, rd, rt</i>	Soustraction	$\text{GPR}[rd] \leftarrow \text{GPR}[rd] - \text{GPR}[rt]$
SB, SH, SW	<i>rt, offset(base)</i>	Ecriture mémoire	$\text{MEM}[\text{GPR}[base] + offset] \leftarrow \text{GPR}[rt]$

A.4 Encodage des instructions MIPS

Le jeu d'instructions MIPS repose sur un encodage de taille fixe, 32-bits. En revanche, trois formats différents d'instructions sont définis : R, I et J. Ces formats sont illustrés à la Figure 10. Il est possible de déterminer le format d'une instruction sur base de son *opcode*, toujours situé dans les 6 bits de poids fort du code.

- Les instructions de **type R** permettent de spécifier 3 registres ainsi qu'un code de fonction (*funct*) et une quantité de décalage (*shamt*). Les instructions `add` et `sll` par exemple utilisent ce format. Toutes ces instructions ont un *opcode* qui vaut `b000000` et sont donc distinguées sur base du champ *funct*.

- Les instructuins de **type I** permettent de spécifier 2 registres ainsi qu’une valeur immédiate de 16-bits (*imm*). Les instructions `addi` et `bgt` par exemple utilisent ce format.
- Les instructions de **type J** permettent de spécifier une valeur immédiate de 26-bits (*target*). Seules les instructions `j` et `jal` utilisent ce format.

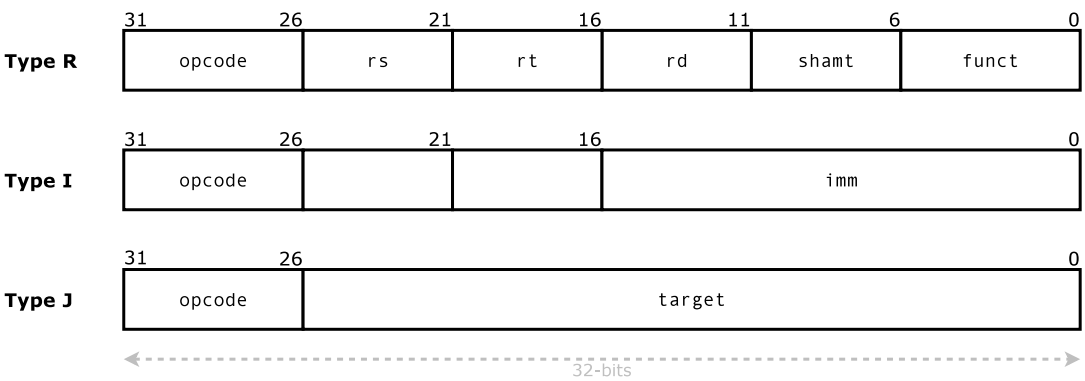


FIGURE 10 – Formats des instructions MIPS.

A.5 Appels système

Le simulateur SPIM permet aux programmes qu’il exécute d’interagir avec l’utilisateur au travers d’une console. Pour cela, il est possible d’effectuer la lecture ou l’écriture de données de et vers la console : entiers, flottants et chaînes de caractères. Ces interactions sont réalisées à l’aide d’« appels systèmes » qu’un programme en langage d’assemblage MIPS peut invoquer. Les appels systèmes supportés par SPIM sont résumés à la Table 7.

TABLE 7 – Liste des appels système supportés par SPIM. Le numéro de l’appel doit être placé dans le registre `v0` avant d’invoquer l’instruction `syscall`.

Service	Numéro appel système	Arguments	Résultats
<code>print int</code>	1	<code>\$a0 = integer</code>	
<code>print float</code>	2	<code>\$f12 = float</code>	
<code>print double</code>	3	<code>\$f12 = double</code>	
<code>print string</code>	4	<code>\$a0 = string</code>	
<code>read int</code>	5		integer (dans <code>\$v0</code>)
<code>read float</code>	6		float (dans <code>\$f0</code>)
<code>read double</code>	7		double (dans <code>\$f0</code>)
<code>read string</code>	8	<code>\$a0 = adresse du buffer,</code> <code>\$a1 = longueur du buffer</code>	
<code>sbrk</code>	9	<code>\$a0 = quantité</code>	adresse (dans <code>\$v0</code>)
<code>exit</code>	10		
<code>print char</code>	11	<code>\$a0 = char</code>	
<code>read char</code>	12		char (dans <code>\$v0</code>)

Pour invoquer un appel système, l’instruction spéciale `syscall` est utilisée. Cette instruction n’a pas d’opérande ; ses arguments sont passés au travers de registres. Le numéro de l’appel système est toujours passé dans le registre `v0`. Chaque appel système transfère ses autres arguments ainsi que ses résultats dans des registres qui varient d’un appel système à l’autre et qui sont documentés à la Table 7 respectivement dans les colonnes « Arguments » et « Résultats ». Par exemple, l’appel système n°1 affiche un entier à la console. L’entier à afficher doit être passé dans le registre `a0`. A l’opposé, l’appel système n°5 lit un entier à partir de la console. L’entier résultant est récupéré dans le registre `v0`.

Remerciements

Merci à V. DHEUR, M. CHARLIER, A. BUYS D. HAUWEELE et V. DUSOLLIER pour leurs remarques sur ce document ou une version antérieure.